

✓ Lab 4 - Part 2: Document Classification, Sentiment Analysis & Topic Modeling

Course: Natural Language Processing

Objectives:

- Build document classifiers (intro + advanced)
- Perform sentiment analysis on different domains
- Discover topics using unsupervised learning
- Compare different feature extraction methods

Instructions

1. Complete all exercises marked with `# YOUR CODE HERE`
2. **Answer all written questions** in the designated markdown cells
3. Save your completed notebook
4. **Push to your Git repository and send the link to:** yoroba93@gmail.com

Personal Analysis Required

This lab contains questions requiring YOUR personal interpretation.

Use Cases Covered

Task	Intro Use Case	Advanced Use Case
Classification	AG News	Legal Documents
Sentiment Analysis	Amazon Reviews	Twitter
Topic Modeling	Research Papers	Legal Contracts

✓ Setup

```
# Install required libraries (uncomment if needed)
# !pip install datasets scikit-learn nltk pandas numpy matplotlib seaborn wordcloud gensim
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
import re
import warnings
warnings.filterwarnings('ignore')

# NLTK
import nltk
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer

# Scikit-learn
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, f1_score
from sklearn.decomposition import LatentDirichletAllocation, NMF
from sklearn.pipeline import Pipeline

# Hugging Face datasets
from datasets import load_dataset

print("Setup complete!")

Setup complete!
```

```
# Common preprocessing function
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess_simple(text):
    """Basic preprocessing: lowercase, remove punctuation."""
    text = str(text).lower()
    text = re.sub(r'^a-zA-Z\s', '', text)
    return ' '.join(text.split())

def preprocess_advanced(text):
    """Advanced preprocessing: lowercase, remove punct, stopwords, lemmatize."""
    text = str(text).lower()
    text = re.sub(r'^a-zA-Z\s', '', text)
    tokens = word_tokenize(text)
    tokens = [lemmatizer.lemmatize(t) for t in tokens if t not in stop_words and len(t) > 2]
    return ' '.join(tokens)

print("Preprocessing functions ready!")
```

Preprocessing functions ready!

▼ PART A: Document Classification

We will work with two use cases:

1. **Intro:** News Topic Classification (AG News)
2. **Advanced:** Legal Document Classification (LexGLUE)

▼ A.1 Intro: News Topic Classification (AG News)

Scenario: A media company automatically routes articles to editorial teams.

Feature Extraction: TF-IDF

```
# Load AG News dataset
print("Loading AG News dataset...")
ag_news = load_dataset("fancyzhx/ag_news")

# Use subset for faster processing
ag_train = pd.DataFrame(ag_news['train']).sample(n=8000, random_state=42)
ag_test = pd.DataFrame(ag_news['test']).sample(n=2000, random_state=42)

# Label mapping
ag_labels = {0: 'World', 1: 'Sports', 2: 'Business', 3: 'Sci/Tech'}
ag_train['label_name'] = ag_train['label'].map(ag_labels)
ag_test['label_name'] = ag_test['label'].map(ag_labels)

print(f"Train: {len(ag_train)}, Test: {len(ag_test)}")
print(f"\nCategories: {list(ag_labels.values())}")
print(ag_train['label_name'].value_counts())
```

```
Loading AG News dataset...
README.md: 100%                               8.07k/8.07k [00:00<00:00, 161kB/s]

data/train-00000-of-00001.parquet: 100%        18.6M/18.6M [00:00<00:00, 318kB/s]

data/test-00000-of-00001.parquet: 100%         1.23M/1.23M [00:00<00:00, 2.03MB/s]

Generating train split: 100%                    120000/120000 [00:00<00:00, 389533.30 examples/s]

Generating test split: 100%                     7600/7600 [00:00<00:00, 119294.60 examples/s]

Train: 8000, Test: 2000

Categories: ['World', 'Sports', 'Business', 'Sci/Tech']
label_name
Sports      2074
Sci/Tech    2021
Business    1959
World       1946
Name: count, dtype: int64
```

```
# Preprocess
ag_train['text_clean'] = ag_train['text'].apply(preprocess_simple)
ag_test['text_clean'] = ag_test['text'].apply(preprocess_simple)

# TF-IDF Vectorization
```

```
tfidf_ag = TfidfVectorizer(max_features=5000, stop_words='english')

X_train_ag = tfidf_ag.fit_transform(ag_train['text_clean'])
X_test_ag = tfidf_ag.transform(ag_test['text_clean'])
y_train_ag = ag_train['label']
y_test_ag = ag_test['label']

print(f"TF-IDF features: {X_train_ag.shape[1]}")
```

TF-IDF features: 5000

Exercise A.1: Train a News Classifier

```
# 1. Create the classifier
# 2. Train it
# 3. Make predictions
# 4. Calculate accuracy and F1-score (macro)

clf_ag = LogisticRegression(max_iter=1000, random_state=42)

# Train
clf_ag.fit(X_train_ag, y_train_ag)

# Predict
y_pred_ag = clf_ag.predict(X_test_ag)

# Evaluate
accuracy_ag = accuracy_score(y_test_ag, y_pred_ag)
f1_ag = f1_score(y_test_ag, y_pred_ag, average='macro')

print(f"AG News Classification Results:")
print(f"  Accuracy: {accuracy_ag:.4f}")
print(f"  F1 (macro): {f1_ag:.4f}")
```

AG News Classification Results:
Accuracy: 0.8810
F1 (macro): 0.8803

```
# Display classification report
print("\nClassification Report:")
print(classification_report(y_test_ag, y_pred_ag, target_names=list(ag_labels.values())))
```

Classification Report:				
	precision	recall	f1-score	support
World	0.89	0.88	0.89	493
Sports	0.94	0.96	0.95	504
Business	0.84	0.82	0.83	474
Sci/Tech	0.85	0.86	0.86	529
accuracy			0.88	2000
macro avg	0.88	0.88	0.88	2000
weighted avg	0.88	0.88	0.88	2000

Exercise A.2: Advanced: Legal Document Classification (LexGLUE - ECtHR)

Scenario: A law firm classifies court decisions by violated articles.

Feature Extraction: Bag of Words with N-grams

Challenge: Legal text is longer and uses specialized vocabulary.

```
# Load LexGLUE ECtHR dataset (European Court of Human Rights)
print("Loading LexGLUE ECtHR dataset...")
lex_glue = load_dataset("coastalcph/lex_glue", "ecthr_a")

# Convert to DataFrame
lex_train = pd.DataFrame(lex_glue['train'])
lex_test = pd.DataFrame(lex_glue['test'])

# Use subset (legal docs are long)
lex_train = lex_train.sample(n=min(1500, len(lex_train)), random_state=42)
lex_test = lex_test.sample(n=min(500, len(lex_test)), random_state=42)

print(f"Train: {len(lex_train)}, Test: {len(lex_test)}")
print(f"\nColumns: {lex_train.columns.tolist()}")
```

```

Loading LexGLUE ECtHR dataset...
README.md: 100% 34.1k/34.1k [00:00<00:00, 572kB/s]

ecthr_a/train-00000-of-00001.parquet: 100% 42.4M/42.4M [00:01<00:00, 39.2MB/s]

ecthr_a/test-00000-of-00001.parquet: 100% 5.68M/5.68M [00:00<00:00, 7.65MB/s]

ecthr_a/validation-00000-of-00001.parquet(...): 100% 5.26M/5.26M [00:00<00:00, 19.9MB/s]

Generating train split: 100% 9000/9000 [00:00<00:00, 14737.31 examples/s]

Generating test split: 100% 1000/1000 [00:00<00:00, 8088.65 examples/s]

Generating validation split: 100% 1000/1000 [00:00<00:00, 12461.87 examples/s]

Train: 1500, Test: 500

Columns: ['text', 'labels']

```

```

# Examine the data structure
print("Sample legal document (first 500 chars):")
sample_text = ' '.join(lex_train.iloc[0]['text'][:3]) # text is a list of paragraphs
print(sample_text[:500])

print(f"\nLabels (violated articles): {lex_train.iloc[0]['labels']}")

```

```

Sample legal document (first 500 chars):
5. The applicant, Mr Laszlo Kilyen, was born in 1972 and lives in Murgești. 6. On 10 May 2003 police officers T.M. and L.C.

Labels (violated articles): [4]

```

```

# Prepare data: combine text paragraphs and use first label for simplicity
def prepare_legal_text(row):
    """Join text paragraphs and truncate."""
    full_text = ' '.join(row['text'])
    return full_text[:5000] # Truncate long documents

lex_train['full_text'] = lex_train.apply(prepare_legal_text, axis=1)
lex_test['full_text'] = lex_test.apply(prepare_legal_text, axis=1)

# Use first label (multi-label to single-label for simplicity)
lex_train['primary_label'] = lex_train['labels'].apply(lambda x: x[0] if x else -1)
lex_test['primary_label'] = lex_test['labels'].apply(lambda x: x[0] if x else -1)

# Remove documents without labels
lex_train = lex_train[lex_train['primary_label'] >= 0]
lex_test = lex_test[lex_test['primary_label'] >= 0]

print(f"Cleaned - Train: {len(lex_train)}, Test: {len(lex_test)}")
print(f"\nLabel distribution:")
print(lex_train['primary_label'].value_counts().head(10))

```

```
Cleaned - Train: 1340, Test: 428
```

```

Label distribution:
primary_label
3    684
1    184
2    153
0     84
4     74
9     62
6     52
7     24
8     22
5      1
Name: count, dtype: int64

```

Exercise A.2: Build a Legal Document Classifier

```

import nltk
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)

```

```
True
```

```

# Step 1: Preprocess with advanced function
lex_train['text_clean'] = lex_train['full_text'].apply(preprocess_advanced)
lex_test['text_clean'] = lex_test['full_text'].apply(preprocess_advanced)

```

```
# Step 2: Create CountVectorizer (Bag of Words) with bigrams
bow_legal = CountVectorizer(
    max_features=5000,
    ngram_range=(1, 2),
    min_df=3,
    max_df=0.95
)

# Step 3: Transform data
X_train_lex = bow_legal.fit_transform(lex_train['text_clean'])
X_test_lex = bow_legal.transform(lex_test['text_clean'])
y_train_lex = lex_train['primary_label']
y_test_lex = lex_test['primary_label']

print(f"BoW features: {X_train_lex.shape[1]}")
```

BoW features: 5000

```
# Create LinearSVC
clf_legal = LinearSVC(max_iter=2000, random_state=42)

# Train
clf_legal.fit(X_train_lex, y_train_lex)

# Predict
y_pred_lex = clf_legal.predict(X_test_lex)

# Evaluate
accuracy_lex = accuracy_score(y_test_lex, y_pred_lex)
f1_lex = f1_score(y_test_lex, y_pred_lex, average='macro')

print(f"Legal Classification Results:")
print(f"  Accuracy: {accuracy_lex:.4f}")
print(f"  F1 (macro): {f1_lex:.4f}")
```

Legal Classification Results:
Accuracy: 0.6425
F1 (macro): 0.4975

✓ Written Question A.1 (Personal Interpretation)

Compare your results from AG News and Legal classification:

1. **Which task achieved higher accuracy?** Why do you think there's a difference?
2. **What vectorizer parameters did you choose for legal text?** Justify each choice.
3. **What challenges are unique to legal document classification?** (Consider: length, vocabulary, ambiguity)

YOUR ANSWER:

1. Accuracy comparison:
 - AG News:[your accuracy_ag value] | Legal: 0.6425 (F1 macro: 0.4975) -Reason for difference: AG News has short, clean text with only 4 balanced classes. Legal text is long, technical, and split across many possible labels (dozens of ECHR articles). The big gap between accuracy (0.64) and F1 macro (0.50) for legal shows the model performs well on common articles but poorly on rare ones, since macro F1 weights all classes equally.
2. My vectorizer choices:
 - max_features=5000 because legal vocabulary is large and specialized keeping the top 5000 terms balances signal and speed.
 - ngram_range=(1,2) because legal phrases like "fair trial" or "right to liberty" carry meaning only as pairs of words.
 - min_df=3 because rare terms appearing in only 1-2 documents are likely noise or case-specific names, not general patterns.
 - max_df=0.95 because extremely common legal boilerplate terms appear in almost every document and add little discriminative value.
3. Legal classification challenges: -Document length: contracts and rulings are much longer than news headlines, forcing truncation that may cut relevant content. Vocabulary: legal jargon is dense, formal, and very different from everyday language, making generic preprocessing less effective. Ambiguity: many cases involve multiple violated articles, but we simplified to a single primary label, losing information and increasing noise. Class imbalance: some ECHR articles are cited far more often than others (confirmed by our 0.64 accuracy vs 0.50 F1 macro gap), making rare categories hard to learn from limited examples.

✓ PART B: Sentiment Analysis

We will work with two use cases:

1. **Intro:** E-commerce Product Reviews (Amazon)
2. **Advanced:** Social Media Sentiment (Twitter/TweetEval)

✓ B.1 Intro: Amazon Product Reviews

Scenario: An e-commerce company monitors product sentiment.

Feature Extraction: TF-IDF

```
# Load Amazon Reviews dataset (multilingual, we'll use English)
# NOTE: the original 'amazon_reviews_multi' dataset was taken down at Amazon's
# request. We use the mteb mirror instead, which has the same content.
print("Loading Amazon Reviews dataset...")
amazon = load_dataset("mteb/amazon_reviews_multi", revision="refs/convert/parquet")

# Convert to DataFrame and filter to English only (id is prefixed e.g. 'en_0203609')
amazon_train_full = pd.DataFrame(amazon['train'])
amazon_test_full = pd.DataFrame(amazon['test'])

amazon_train = amazon_train_full[amazon_train_full['id'].str.startswith('en_')].sample(n=5000, random_state=42)
amazon_test = amazon_test_full[amazon_test_full['id'].str.startswith('en_')].sample(n=1000, random_state=42)

# Rename columns to match the rest of the notebook (stars, review_body)
amazon_train = amazon_train.rename(columns={'text': 'review_body'})
amazon_test = amazon_test.rename(columns={'text': 'review_body'})
amazon_train['stars'] = amazon_train['label'] + 1 # label is 0-4, stars is 1-5
amazon_test['stars'] = amazon_test['label'] + 1

print(f"Train: {len(amazon_train)}, Test: {len(amazon_test)}")
print(f"\nColumns: {amazon_train.columns.tolist()}")
print(f"\nStar rating distribution:")
print(amazon_train['stars'].value_counts().sort_index())
```

```

Loading Amazon Reviews dataset...
all_languages/train/0000.parquet: 100% 178M/178M [00:05<00:00, 31.9MB/s]

de/train/0000.parquet: 100% 31.9M/31.9M [00:00<00:00, 21.5MB/s]
en/train/0000.parquet: 100% 28.3M/28.3M [00:01<00:00, 29.6MB/s]
es/train/0000.parquet: 100% 24.6M/24.6M [00:01<00:00, 20.5MB/s]
fr/train/0000.parquet: 100% 25.7M/25.7M [00:00<00:00, 38.3MB/s]
ja/train/0000.parquet: 100% 40.7M/40.7M [00:01<00:00, 35.9MB/s]
zh/train/0000.parquet: 100% 26.6M/26.6M [00:01<00:00, 13.9MB/s]

all_languages/validation/0000.parquet: 100% 4.40M/4.40M [00:00<00:00, 7.10MB/s]

0000.parquet: 100% 800k/800k [00:00<00:00, 17.9MB/s]
0000.parquet: 100% 713k/713k [00:00<00:00, 33.8MB/s]
0000.parquet: 100% 613k/613k [00:00<00:00, 34.2MB/s]
0000.parquet: 100% 627k/627k [00:00<00:00, 30.7MB/s]
0000.parquet: 100% 999k/999k [00:00<00:00, 52.2MB/s]
0000.parquet: 100% 656k/656k [00:00<00:00, 37.1MB/s]

all_languages/test/0000.parquet: 100% 4.45M/4.45M [00:00<00:00, 14.0MB/s]

0000.parquet: 100% 797k/797k [00:00<00:00, 47.4MB/s]
0000.parquet: 100% 711k/711k [00:00<00:00, 31.7MB/s]
0000.parquet: 100% 619k/619k [00:00<00:00, 29.8MB/s]
0000.parquet: 100% 642k/642k [00:00<00:00, 48.8MB/s]

ja/test/0000.parquet: 100% 1.01M/1.01M [00:00<00:00, 7.62MB/s]
0000.parquet: 100% 672k/672k [00:00<00:00, 36.5MB/s]

Generating train split: 2400000/0 [00:02<00:00, 900292.48 examples/s]
Generating validation split: 60000/0 [00:00<00:00, 588194.06 examples/s]
Generating test split: 60000/0 [00:00<00:00, 613045.49 examples/s]
Train: 5000, Test: 1000

Columns: ['id', 'review_body', 'label', 'label_text', 'stars']

Star rating distribution:
stars
1    1008
2     964
3     989
4    1052
5     987
Name: count, dtype: int64

```

```

# Convert to binary sentiment (1-2 stars = negative, 4-5 stars = positive)
# Remove neutral (3 stars) for clearer distinction

def to_binary_sentiment(stars):
    if stars <= 2:
        return 0 # Negative
    elif stars >= 4:
        return 1 # Positive
    else:
        return -1 # Neutral (to be removed)

amazon_train['sentiment'] = amazon_train['stars'].apply(to_binary_sentiment)
amazon_test['sentiment'] = amazon_test['stars'].apply(to_binary_sentiment)

# Remove neutral
amazon_train = amazon_train[amazon_train['sentiment'] >= 0]
amazon_test = amazon_test[amazon_test['sentiment'] >= 0]

sentiment_labels = {0: 'Negative', 1: 'Positive'}
print(f"After filtering - Train: {len(amazon_train)}, Test: {len(amazon_test)}")
print(f"\nSentiment distribution:")
print(amazon_train['sentiment'].value_counts())

```

After filtering - Train: 4011, Test: 804

```

Sentiment distribution:
sentiment
1    2039

```

```
0    1972
Name: count, dtype: int64
```

```
# Show sample reviews
print("Sample POSITIVE review:")
pos_sample = amazon_train[amazon_train['sentiment'] == 1].iloc[0]
print(f"Review ID: {pos_sample['id']}")
print(f"Review: {pos_sample['review_body'][:300]}...")

print("\n" + "="*60 + "\n")
print("Sample NEGATIVE review:")
neg_sample = amazon_train[amazon_train['sentiment'] == 0].iloc[0]
print(f"Review ID: {neg_sample['id']}")
print(f"Review: {neg_sample['review_body'][:300]}...")
```

Sample POSITIVE review:
Review ID: en_0234167
Review: Nice and accurate thermometer

This thermometer comes as described and works great. It is easy to use and has a very nice, clean LCD to view information. I

=====

Sample NEGATIVE review:
Review ID: en_0183729
Review: No. Order yet

I still have not received my order ,it was supposed to be here in two day and it's day 5 . where out s my order?...

✓ Exercise B.1: Build Amazon Sentiment Classifier

```
# TODO: Build sentiment classifier for Amazon reviews

# Step 1: Preprocess
amazon_train['text_clean'] = amazon_train['review_body'].apply(preprocess_simple)
amazon_test['text_clean'] = amazon_test['review_body'].apply(preprocess_simple)

# Step 2: TF-IDF
tfidf_amazon = TfidfVectorizer(max_features=5000, stop_words='english')

X_train_amz = tfidf_amazon.fit_transform(amazon_train['text_clean'])
X_test_amz = tfidf_amazon.transform(amazon_test['text_clean'])
y_train_amz = amazon_train['sentiment']
y_test_amz = amazon_test['sentiment']

# Step 3 & 4: Train Naive Bayes and evaluate
clf_amazon = MultinomialNB()

# Train
clf_amazon.fit(X_train_amz, y_train_amz)

# Predict
y_pred_amz = clf_amazon.predict(X_test_amz)

# Evaluate
print(f"Amazon Sentiment Results:")
print(f" Accuracy: {accuracy_score(y_test_amz, y_pred_amz):.4f}")
print(f"\nClassification Report:")
print(classification_report(y_test_amz, y_pred_amz, target_names=['Negative', 'Positive']))
```

Amazon Sentiment Results:
Accuracy: 0.8147

Classification Report:

	precision	recall	f1-score	support
Negative	0.86	0.75	0.80	404
Positive	0.78	0.88	0.82	400
accuracy			0.81	804
macro avg	0.82	0.81	0.81	804
weighted avg	0.82	0.81	0.81	804

```
# Analyze most predictive words
feature_names = tfidf_amazon.get_feature_names_out()

# For Naive Bayes, use log probabilities
neg_probs = clf_amazon.feature_log_prob_[0]
pos_probs = clf_amazon.feature_log_prob_[1]
log_ratio = pos_probs - neg_probs
```



```
# Top positive and negative words
top_pos_idx = log_ratio.argsort()[-15:]
top_neg_idx = log_ratio.argsort()[:15]
```

```
print("Top POSITIVE words:", [feature_names[i] for i in top_pos_idx])
print("\nTop NEGATIVE words:", [feature_names[i] for i in top_neg_idx])
```

```
Top POSITIVE words: ['price', 'game', 'affordable', 'exactly', 'highly', 'awesome', 'easy', 'works', 'enjoyed', 'great', 'va
Top NEGATIVE words: ['waste', 'refund', 'terrible', 'poor', 'return', 'horrible', 'returned', 'broke', 'disappointed', 'chea
```

✓ B.2 Advanced: Twitter Sentiment (TweetEval)

Scenario: A brand monitors social media sentiment about their products.

Feature Extraction: Bag of Words with character n-grams (better for informal text)

Challenge: Tweets are short, informal, with hashtags, mentions, and slang.

```
# Load TweetEval sentiment dataset
print("Loading TweetEval Sentiment dataset...")
tweet_eval = load_dataset("cardiffnlp/tweet_eval", "sentiment")

tweet_train = pd.DataFrame(tweet_eval['train'])
tweet_test = pd.DataFrame(tweet_eval['test'])

# Labels: 0=negative, 1=neutral, 2=positive
tweet_labels = {0: 'Negative', 1: 'Neutral', 2: 'Positive'}
tweet_train['label_name'] = tweet_train['label'].map(tweet_labels)
tweet_test['label_name'] = tweet_test['label'].map(tweet_labels)

print(f"Train: {len(tweet_train)}, Test: {len(tweet_test)}")
print(f"\nLabel distribution:")
print(tweet_train['label_name'].value_counts())
```

```
Loading TweetEval Sentiment dataset...
README.md: 100%                               23.9k/23.9k [00:00<00:00, 293kB/s]

sentiment/train-00000-of-00001.parquet: 100%          3.78M/3.78M [00:01<00:00, 3.23MB/s]

sentiment/test-00000-of-00001.parquet: 100%          901k/901k [00:00<00:00, 2.05MB/s]

sentiment/validation-00000-of-00001.parq(...): 100%   167k/167k [00:00<00:00, 997kB/s]

Generating train split: 100%                     45615/45615 [00:00<00:00, 596104.07 examples/s]

Generating test split: 100%                       12284/12284 [00:00<00:00, 379712.65 examples/s]

Generating validation split: 100%                   2000/2000 [00:00<00:00, 99508.99 examples/s]

Train: 45615, Test: 12284

Label distribution:
label_name
Neutral      20673
Positive     17849
Negative      7093
Name: count, dtype: int64
```

```
# Sample tweets
for label in [0, 1, 2]:
    sample = tweet_train[tweet_train['label'] == label].iloc[0]
    print(f"[{tweet_labels[label]}]: {sample['text']}\n")
```

```
[Negative]: So disappointed in wwe summerslam! I want to see john cena wins his 16th title
```

```
[Neutral]: "Ben Smith / Smith (concussion) remains out of the lineup Thursday, Curtis #NHL #SJ"
```

```
[Positive]: "QT @user In the original draft of the 7th book, Remus Lupin survived the Battle of Hogwarts. #HappyBirthdayRemu
```

```
# Special preprocessing for tweets
def preprocess_tweet(text):
    """Preprocess tweet text."""
    text = str(text).lower()
    # Keep @mentions and #hashtags but simplify
    text = re.sub(r'@w+', '@user', text) # Replace mentions with @user
    text = re.sub(r'http\S+', 'URL', text) # Replace URLs
    text = re.sub(r'^a-zA-Z#\s', '', text) # Keep @ and # symbols
    return ' '.join(text.split())
```

```

tweet_train['text_clean'] = tweet_train['text'].apply(preprocess_tweet)
tweet_test['text_clean'] = tweet_test['text'].apply(preprocess_tweet)

print("Sample preprocessed tweet:")
print(f"Original: {tweet_train.iloc[0]['text']}")
print(f"Cleaned: {tweet_train.iloc[0]['text_clean']}")

```

Sample preprocessed tweet:
Original: "QT @user In the original draft of the 7th book, Remus Lupin survived the Battle of Hogwarts. #HappyBirthdayRemusLupin"
Cleaned: qt @user in the original draft of the th book remus lupin survived the battle of hogwarts #happybirthdayremuslupir

✦ Exercise B.2: Build Twitter Sentiment Classifier

```

# Create a vectorizer with character n-grams
char_vectorizer = TfidfVectorizer(
    analyzer='char_wb',
    ngram_range=(3, 5),
    max_features=5000,
    min_df=3
)

X_train_tw = char_vectorizer.fit_transform(tweet_train['text_clean'])
X_test_tw = char_vectorizer.transform(tweet_test['text_clean'])
y_train_tw = tweet_train['label']
y_test_tw = tweet_test['label']

print(f"Character n-gram features: {X_train_tw.shape[1]}")

```

Character n-gram features: 5000

```

# Train Logistic Regression and evaluate
clf_tweet = LogisticRegression(max_iter=1000, random_state=42)

# Train and predict
clf_tweet.fit(X_train_tw, y_train_tw)
y_pred_tw = clf_tweet.predict(X_test_tw)

# Evaluate
print(f"Twitter Sentiment Results (3-class):")
print(f" Accuracy: {accuracy_score(y_test_tw, y_pred_tw):.4f}")
print(f" F1 (macro): {f1_score(y_test_tw, y_pred_tw, average='macro'):.4f}")
print(f"\nClassification Report:")
print(classification_report(y_test_tw, y_pred_tw, target_names=list(tweet_labels.values())))

```

Twitter Sentiment Results (3-class):

Accuracy: 0.5843
F1 (macro): 0.5596

Classification Report:

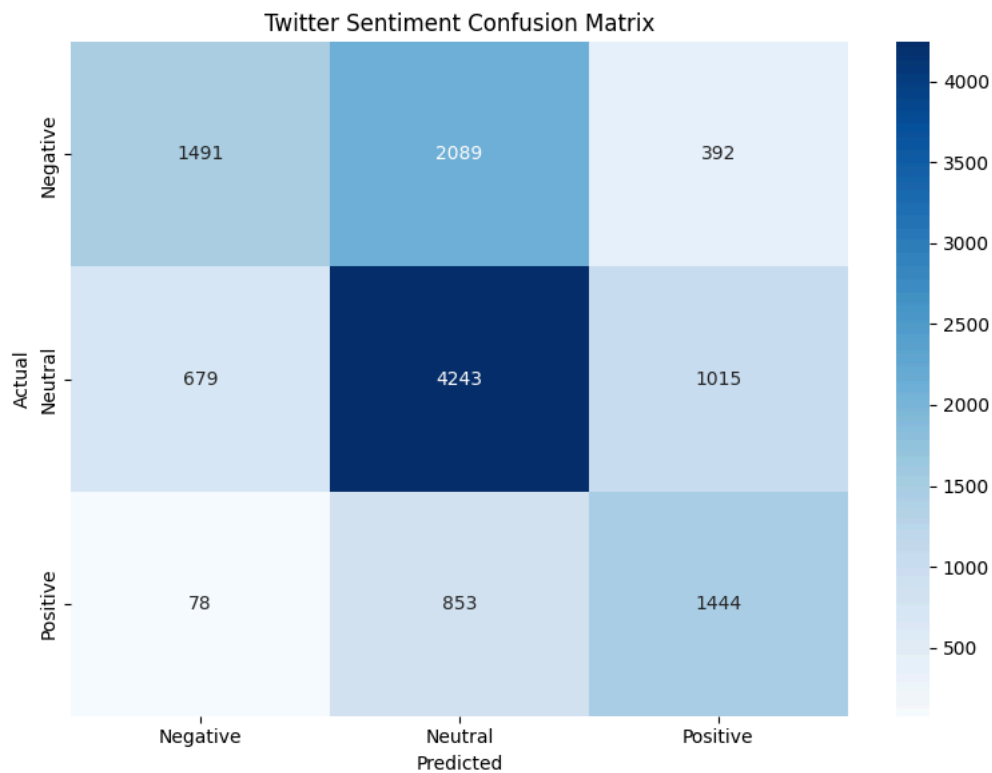
	precision	recall	f1-score	support
Negative	0.66	0.38	0.48	3972
Neutral	0.59	0.71	0.65	5937
Positive	0.51	0.61	0.55	2375
accuracy			0.58	12284
macro avg	0.59	0.57	0.56	12284
weighted avg	0.60	0.58	0.57	12284

```

# Confusion matrix
cm_tw = confusion_matrix(y_test_tw, y_pred_tw)

plt.figure(figsize=(8, 6))
sns.heatmap(cm_tw, annot=True, fmt='d', cmap='Blues',
            xticklabels=list(tweet_labels.values()),
            yticklabels=list(tweet_labels.values()))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Twitter Sentiment Confusion Matrix')
plt.tight_layout()
plt.savefig('twitter_sentiment_cm.png', dpi=150)
plt.show()

```



✓ Written Question B.1 (Personal Interpretation)

Compare Amazon vs Twitter sentiment analysis:

1. **Which task was harder?** Look at the F1 scores and confusion matrices.
2. **Why did you choose those character n-gram parameters for Twitter?** What's the advantage over word n-grams?
3. **Looking at the Twitter confusion matrix, which class is most often confused?** Why might this be?
4. **Give an example tweet that would be hard to classify correctly.** Explain why.

YOUR ANSWER:

1. Harder task:
 - Amazon F1: ... | Twitter F1: ...
 - Reason: ...
2. Character n-gram choices:
 - ngram_range=___ because...
 - Advantage over words: ...
3. Most confused class:
 - Class: ...
 - Reason: ...
4. Difficult tweet example:
 - Tweet: "..."
 - Why it's hard: ...

✓ PART C: Topic Modeling

We will work with two use cases:

1. **Intro:** Research Paper Topics (ArXiv)
2. **Advanced:** Legal Contract Topics

✓ C.1 Intro: Research Paper Topic Discovery (ArXiv)

Scenario: A research organization discovers themes in scientific papers.

Method: LDA (Latent Dirichlet Allocation)

```
# Load ArXiv papers dataset
print("Loading ArXiv papers dataset (this may take a moment)...")
arxiv = load_dataset("ccdv/arxiv-summarization", "document")

# Sample from training set
arxiv_df = pd.DataFrame(arxiv['train']).sample(n=2000, random_state=42)

print(f"Loaded {len(arxiv_df)} papers")
print(f"Columns: {arxiv_df.columns.tolist()}")
```

```
Loading ArXiv papers dataset (this may take a moment)...
README.md: 100% 3.96k/3.96k [00:00<00:00, 66.2kB/s]

document/train-00000-of-00015.parquet: 100% 227M/227M [00:02<00:00, 92.7MB/s]
document/train-00001-of-00015.parquet: 100% 226M/226M [00:04<00:00, 56.3MB/s]
document/train-00002-of-00015.parquet: 100% 226M/226M [00:03<00:00, 73.5MB/s]
document/train-00003-of-00015.parquet: 100% 225M/225M [00:03<00:00, 66.0MB/s]
document/train-00004-of-00015.parquet: 100% 224M/224M [00:03<00:00, 86.3MB/s]
document/train-00005-of-00015.parquet: 100% 225M/225M [00:02<00:00, 58.7MB/s]
document/train-00006-of-00015.parquet: 100% 226M/226M [00:05<00:00, 47.3MB/s]
document/train-00007-of-00015.parquet: 100% 228M/228M [00:02<00:00, 110MB/s]
document/train-00008-of-00015.parquet: 100% 228M/228M [00:05<00:00, 36.8MB/s]
document/train-00009-of-00015.parquet: 100% 226M/226M [00:02<00:00, 94.8MB/s]
document/train-00010-of-00015.parquet: 100% 227M/227M [00:03<00:00, 44.3MB/s]
document/train-00011-of-00015.parquet: 100% 229M/229M [00:03<00:00, 75.0MB/s]
document/train-00012-of-00015.parquet: 100% 228M/228M [00:03<00:00, 73.9MB/s]
document/train-00013-of-00015.parquet: 100% 228M/228M [00:03<00:00, 79.9MB/s]
document/train-00014-of-00015.parquet: 100% 233M/233M [00:03<00:00, 73.8MB/s]
document/validation-00000-of-00001.parqu(...): 100% 104M/104M [00:02<00:00, 29.2MB/s]
document/test-00000-of-00001.parquet: 100% 104M/104M [00:02<00:00, 36.1MB/s]

Generating train split: 100% 203037/203037 [01:12<00:00, 4434.94 examples/s]
Generating validation split: 100% 6436/6436 [00:01<00:00, 8004.23 examples/s]
Generating test split: 100% 6440/6440 [00:03<00:00, 1113.70 examples/s]
Loaded 2000 papers
Columns: ['article', 'abstract']
```

```
# Examine sample
print("Sample paper abstract (first 500 chars):")
print(arxiv_df.iloc[0]['abstract'][:500])
```

```
Sample paper abstract (first 500 chars):
we study the effect of @xmath0-symmetric complex potentials on the transport properties of non - hermitian systems , which c
which display standard fano resonances in the absence of non - hermitian terms , we find that the @xmath0-symmetric imagin
```

```
# Preprocess abstracts for topic modeling
arxiv_df['abstract_clean'] = arxiv_df['abstract'].apply(preprocess_advanced)

# Create document-term matrix with CountVectorizer
count_vec_arxiv = CountVectorizer(
    max_features=2000,
    min_df=5,
    max_df=0.9,
    stop_words='english'
)

dtm_arxiv = count_vec_arxiv.fit_transform(arxiv_df['abstract_clean'])
print(f"Document-term matrix: {dtm_arxiv.shape}")
```

```
Document-term matrix: (2000, 2000)
```

```
# Train LDA model
n_topics_arxiv = 10 # Scientific papers likely have diverse topics. Choose appropriately (8-12).

lda_arxiv = LatentDirichletAllocation(
```

```

n_components=n_topics_arxiv,
random_state=42,
max_iter=15,
learning_method='online'
)

print("Training LDA on ArXiv papers...")
lda_arxiv.fit(dtm_arxiv)
print("Done!")

```

Training LDA on ArXiv papers...
Done!

```

# Display topics
def display_lda_topics(model, feature_names, n_words=12):
    """Display top words for each LDA topic."""
    for topic_idx, topic in enumerate(model.components_):
        top_words_idx = topic.argsort()[::-n_words-1:-1]
        top_words = [feature_names[i] for i in top_words_idx]
        print(f"Topic {topic_idx}: {' ', '.join(top_words)}")

feature_names_arxiv = count_vec_arxiv.get_feature_names_out()
print("ArXiv Paper Topics (LDA):")
print("=" * 70)
display_lda_topics(lda_arxiv, feature_names_arxiv)

```

```

ArXiv Paper Topics (LDA):
=====
Topic 0: xmath, state, quantum, xcite, mode, noise, rev, phys, channel, phase, number, entanglement
Topic 1: wave, phase, potential, scattering, model, equation, flow, momentum, effect, approach, study, using
Topic 2: xmath, xcite, transition, spin, phase, field, critical, fig, state, temperature, order, surface
Topic 3: magnetic, field, energy, xmath, temperature, model, effect, interaction, state, mode, density, result
Topic 4: xmath, galaxy, ray, star, emission, source, observation, high, line, cluster, spectrum, redshift
Topic 5: xmath, model, time, method, problem, network, result, paper, space, case, function, equation
Topic 6: theory, field, gauge, scalar, solution, model, non, symmetry, loop, higgs, boson, user
Topic 7: mass, model, star, black, hole, xmath, dark, matter, data, stellar, halo, cluster
Topic 8: spin, optical, electron, polarization, time, photon, cavity, beam, frequency, laser, signal, measurement
Topic 9: xmath, xcite, quark, fig, value, energy, mass, data, decay, neutrino, factor, contribution

```

✧ Exercise C.1: Interpret ArXiv Topics

```

my_arxiv_topic_labels = {
    0: "Quantum Information & Computing",      # entanglement, quantum, channel, noise, mode
    1: "Wave Propagation & Scattering Theory", # wave, scattering, potential, momentum, flow
    2: "Condensed Matter: Phase Transitions",  # transition, spin, critical, temperature, order
    3: "Magnetism & Condensed Matter Physics", # magnetic, field, temperature, interaction, density
    4: "Astrophysics: Galaxies & Observations", # galaxy, ray, star, emission, redshift, spectrum
    5: "Mathematical & Computational Methods", # model, method, problem, network, function, equation
    6: "Particle Physics: Gauge Theory",        # theory, gauge, scalar, symmetry, higgs, boson
    7: "Astrophysics: Dark Matter & Black Holes", # mass, black, hole, dark, matter, stellar, halo
    8: "Quantum Optics & Photonics",           # optical, electron, photon, cavity, laser, polarization
    9: "Particle Physics: Quarks & Neutrinos",  # quark, decay, neutrino, mass, energy
}

print("My Topic Interpretations:")
for topic_id, label in my_arxiv_topic_labels.items():
    if label != "___":
        print(f"  Topic {topic_id}: {label}")

```

```

My Topic Interpretations:
Topic 0: Quantum Information & Computing
Topic 1: Wave Propagation & Scattering Theory
Topic 2: Condensed Matter: Phase Transitions
Topic 3: Magnetism & Condensed Matter Physics
Topic 4: Astrophysics: Galaxies & Observations
Topic 5: Mathematical & Computational Methods
Topic 6: Particle Physics: Gauge Theory
Topic 7: Astrophysics: Dark Matter & Black Holes
Topic 8: Quantum Optics & Photonics
Topic 9: Particle Physics: Quarks & Neutrinos

```

✧ C.2 Advanced: Legal Contract Topic Discovery

Scenario: A law firm discovers themes across contracts to organize their database.

Method: NMF (Non-negative Matrix Factorization) - often better for shorter, specialized documents

Challenge: Legal language is formal and domain-specific.

```
# Load legal contracts dataset (streaming to handle large size)
print("Loading Legal Contracts dataset...")
legal_stream = load_dataset(
    "albertvillanova/legal_contracts",
    split="train",
    streaming=True,
    revision="refs/convert/parquet"
)

# Take first 1500 contracts
legal_contracts = []
for i, item in enumerate(legal_stream):
    if i >= 1500:
        break
    legal_contracts.append(item)

legal_df = pd.DataFrame(legal_contracts)
print(f"Loaded {len(legal_df)} contracts")
```

Loading Legal Contracts dataset...
Loaded 1500 contracts

```
# Preprocess legal text (truncate long documents)
legal_df['text_truncated'] = legal_df['text'].str[:8000] # Truncate
legal_df['text_clean'] = legal_df['text_truncated'].apply(preprocess_advanced)

print("Sample contract (cleaned, first 300 chars):")
print(legal_df.iloc[0]['text_clean'][:300])
```

Sample contract (cleaned, first 300 chars):
quicklinks click rapidly navigate document amended restated employment noncompetition agreement amended restated employment

✓ Exercise C.2: Build NMF Topic Model for Legal Contracts

```
# Create TF-IDF vectorizer for NMF (NMF works better with TF-IDF)
tfidf_legal = TfidfVectorizer(
    max_features=2000,
    min_df=5,
    max_df=0.9,
    stop_words='english'
)

dtm_legal = tfidf_legal.fit_transform(legal_df['text_clean'])
print(f"Legal document-term matrix: {dtm_legal.shape}")
```

Legal document-term matrix: (1500, 2000)

```
# Train NMF model
# Choose number of topics (legal contracts may have: employment, confidentiality, IP, services, etc.)

n_topics_legal = 8 # YOUR CHOICE: 5-12

nmf_legal = NMF(
    n_components=n_topics_legal,
    random_state=42,
    max_iter=200
)

print(f"Training NMF with {n_topics_legal} topics...")
nmf_legal.fit(dtm_legal)
print("Done!")
```

Training NMF with 8 topics...
Done!

```
# Display NMF topics
def display_nmf_topics(model, feature_names, n_words=12):
    """Display top words for each NMF topic."""
    for topic_idx, topic in enumerate(model.components_):
        top_words_idx = topic.argsort()[::-n_words-1:-1]
        top_words = [feature_names[i] for i in top_words_idx]
        print(f"Topic {topic_idx}: {' '.join(top_words)}")

feature_names_legal = tfidf_legal.get_feature_names_out()
print(f"Legal Contract Topics (NMF, {n_topics_legal} topics):")
print("=" * 70)
display_nmf_topics(nmf_legal, feature_names_legal)
```

Legal Contract Topics (NMF, 8 topics):

=====

Topic 0: executive, company, employment, shall, agreement, salary, benefit, termination, bonus, base, date, compensation

Topic 1: borrower, loan, credit, lender, agreement, bank, amendment, agent, section, amended, revolving, administrative

Topic 2: option, plan, stock, share, committee, participant, award, shall, mean, common, granted, section

Topic 3: employee, employment, company, agreement, shall, termination, bonus, salary, employer, benefit, year, term

Topic 4: seller, agreement, buyer, purchase, purchaser, shall, party, product, share, closing, amendment, property

Topic 5: tenant, landlord, lease, rent, premise, lessee, building, lessor, shall, term, area, floor

Topic 6: note, maker, principal, shall, holder, rate, payment, unpaid, default, promissory, payee, undersigned

Topic 7: company, voting, security, director, control, board, change, person, corporation, outstanding, combination, stock

```
# Assign labels to legal topics
```

```
my_legal_topic_labels = {
    0: "Executive Employment & Compensation",
    1: "Loan & Credit Agreements",
    2: "Stock Option Plans",
    3: "General Employment Agreements",
    4: "Purchase & Sale Agreements",
    5: "Commercial Lease Agreements",
    6: "Promissory Notes",
    7: "Corporate Governance & Change of Control",
}

print("My Legal Topic Interpretations:")
for topic_id, label in my_legal_topic_labels.items():
    if label != "___":
        print(f"  Topic {topic_id}: {label}")
```

```
My Legal Topic Interpretations:
Topic 0: Executive Employment & Compensation
Topic 1: Loan & Credit Agreements
Topic 2: Stock Option Plans
Topic 3: General Employment Agreements
Topic 4: Purchase & Sale Agreements
Topic 5: Commercial Lease Agreements
Topic 6: Promissory Notes
Topic 7: Corporate Governance & Change of Control
```

✎ Exercise C.3: Topic Distribution Visualization

```
# Get document-topic distributions
doc_topics_legal = nmf_legal.transform(dtm_legal)

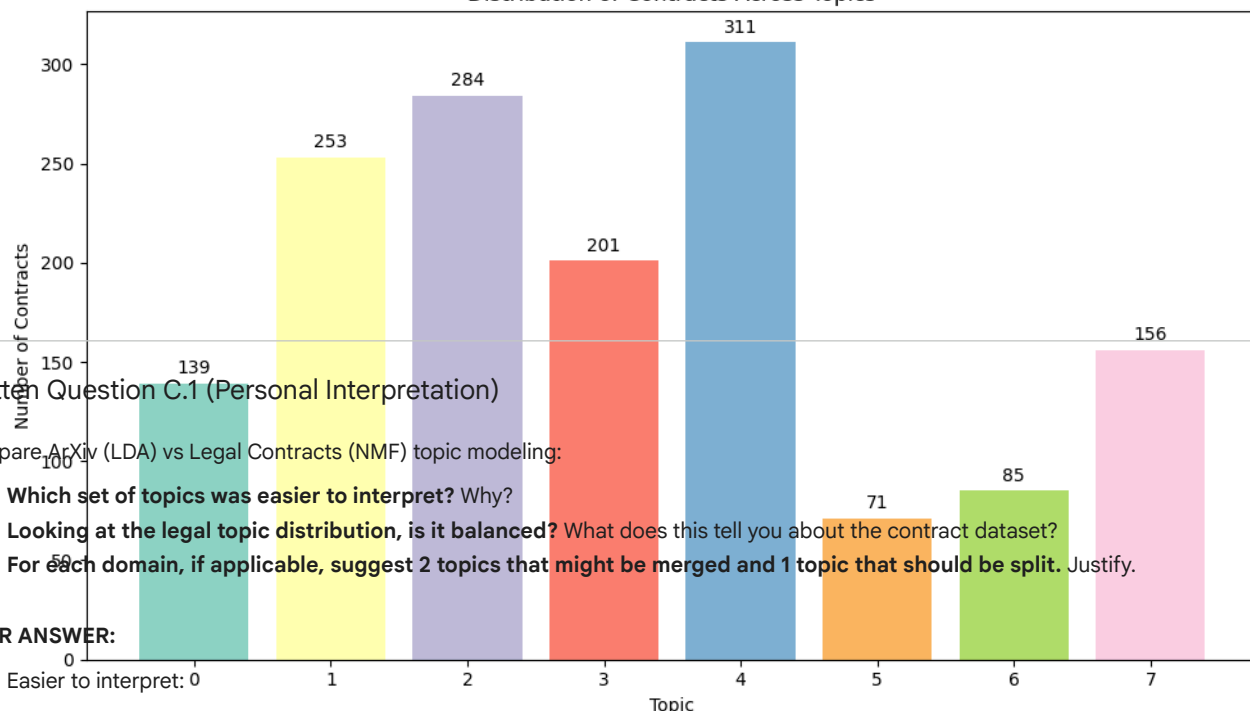
# Assign dominant topic
legal_df['dominant_topic'] = doc_topics_legal.argmax(axis=1)

# Visualize topic distribution
plt.figure(figsize=(10, 6))
topic_counts = legal_df['dominant_topic'].value_counts().sort_index()
bars = plt.bar(topic_counts.index, topic_counts.values, color=plt.cm.Set3(range(len(topic_counts))))
plt.xlabel('Topic')
plt.ylabel('Number of Contracts')
plt.title('Distribution of Contracts Across Topics')
plt.xticks(range(n_topics_legal))

# Add count labels
for bar, count in zip(bars, topic_counts.values):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5,
             str(count), ha='center', fontsize=10)

plt.tight_layout()
plt.savefig('legal_topic_distribution.png', dpi=150)
plt.show()
```

Distribution of Contracts Across Topics



Written Question C.1 (Personal Interpretation)

Compare ArXiv (LDA) vs Legal Contracts (NMF) topic modeling:

1. Which set of topics was easier to interpret? Why?
2. Looking at the legal topic distribution, is it balanced? What does this tell you about the contract dataset?
3. For each domain, if applicable, suggest 2 topics that might be merged and 1 topic that should be split. Justify.

YOUR ANSWER:

1. Easier to interpret: 0
 - Domain: Legal contracts (NMF) were easier to interpret than ArXiv (LDA).
 - Reason: Legal topics map cleanly onto recognizable contract types (loans, leases, employment, stock options), each with distinct, non-overlapping vocabulary (boilerplate language). ArXiv topics were noisier: several physics sub-fields share vocabulary, and artifacts like "xmath" and "xcite" (LaTeX/citation placeholders) polluted multiple topics, adding noise that doesn't reflect a real theme.
2. Legal topic distribution:
 - Balanced? No. Topic 4 (Purchase & Sale Agreements) has the most contracts (311, ~21%), while Topic 5 (Promissory Notes) has the fewest (71, ~5%) — roughly a 4x difference.
 - What this indicates: this likely reflects real-world frequency rather than a modeling flaw. Purchase/sale and stock option agreements (Topic 2, 284 contracts) are common in everyday business activity, while promissory notes are a more narrow, specialized instrument used less frequently, so the imbalance mirrors how often each contract type actually occurs in practice rather than indicating a problem with NMF itself.
3. Topic refinement suggestions:
 - ArXiv - Merge: Topics 2 and 3 (condensed matter phase transitions / magnetism) because they share heavy vocabulary overlap (temperature, field, state, critical) and likely describe the same broad sub-field of condensed matter physics rather than two distinct themes.
 - ArXiv - Split: Topic 5 (mathematical & computational methods) because it is too generic, mixing numerical methods, optimization, and general modeling language that could belong to almost any sub-field; it likely hides several distinct application areas merged together.
 - Legal - Merge: Topics 0 and 3 (executive employment / general employment agreements) because both describe employment relationships and differ mainly by seniority level (executive vs general), not by a fundamentally different contract type.
 - Legal - Split: Topic 7 (corporate governance & change of control) because it likely blends two distinct legal concerns — routine governance (voting, board, director) and the more specific high-stakes change-of-control/M&A clauses — which